Open in Colab

`range` Objects

In a typical `for` statement we need a counter, usually starting with `0` and incremented by `1` in each step until it reaches to some number. This is written in Python with `range` object.

In [ ]:
```
for i in range(0,6,2): print(i)
```

```
0
2
4
```

Here, `range(0,6,2)` is a generator i.e. it generates all the numbers starting from `0` until but not including `6` stepping by `2`.

`step` argument of `range` can also be a negative number to count in decreasing order:

In [ ]:
```
for i in range(6,0,-2): print(i)
```

```
6
4
2
```

Here, step and start arguments are optional and defaults to `step=1` and `start=0`.

If you just write `range(2**256)` without really using it then it will not do anything. All the numbers are generated on demand.

In [ ]:
```
range(2**256)
```

Out[ ]:
```
range(0, 115792089237316195423570985008687907853269984665640564039457584007913129639936)
```

Usage:

In [ ]:
```
print(range.__doc__)
```

```
range(stop) -> range object
range(start, stop[, step]) -> range object

Return an object that produces a sequence of integers from start (inclusive)
to stop (exclusive) by step.  range(i, j) produces i, i+1, i+2, ..., j-1.
start defaults to 0, and stop is omitted!  range(4) produces 0, 1, 2, 3.
These are exactly the valid indices for a list of 4 elements.
When step is given, it specifies the increment (or decrement).
```

Similar to `range` we can define out own generaters as follows:

Generators

Functions are used to return a single value (object) on each call. In contrast generators can be used to generate multiple values sequently.

Recall that `return` statement terminates the function evaluation and returns a value. In generators `yield` statement only returns a value without terminating the function.

Generators can be defined with `def` statements like functions.

In [ ]:
```
def fibonacci(a=0,b=1):
  while True:
    yield a
    a,b = b,a+b
```

But calling a generator does not run that. Insted it creates a generator object that can be used as itarable.

In [ ]:
```
fibs = fibonacci()
fibs
```

Out[ ]:
```
<generator object fibonacci at 0x7ade31706740>
```

Then you can use `next` function to get the next item.

In [ ]:
```
next(fibs)
```

Out[ ]:
```
0
```

In [ ]:
```
next(fibs)
```

Out[ ]:
```
1
```

In [ ]:
```
fibs = fibonacci()
```

In [ ]:
```
for _ in range(10): print(next(fibs),end=' ')
```

```
0 1 1 2 3 5 8 13 21 34
```

Generators saves it state so can continue running with subsequent `next` calls.

In [ ]:
```
for _ in range(10):print(next(fibs),end=' ')
```

```
55 89 144 233 377 610 987 1597 2584 4181
```

To reinitialize the generator:

In [ ]:
```
fibs = fibonacci()
for i in range(10):print(next(fibs),end=' ')
```

```
0 1 1 2 3 5 8 13 21 34
```

The advantage of using the generators instead of functions is that functions must finish all its jobs before `return`. But generators don't need this. Because it resumes/pauses the execution on each `yield` statement.

Therefore generators can be used to generate infinite values.

Of course they can be finite as well:

In [ ]:
```
def gcd_helper(a,b):
  """This will generate a decreasing sequnce of numbers.
  The last number will be the `gcd` of a and b
  """
  yield a
  while b:
    yield b
    a, b = b, a % b
```

Then you can use generators for generating finite iterables. And convert them to other iterables.

In [ ]:
```
tuple(gcd_helper(1112,2116))
```

Out[ ]:
```
(1112, 2116, 1112, 1004, 108, 32, 12, 8, 4)
```

`return` In Generators

Like functions, you can terminate a generator with `return` statement.

In [ ]:
```
def gcd_helper(a,b):
  """This will generate a decreasing sequnce of numbers.
  The last number will be the `gcd` of a and b
  """
  if a < 0 or b < 0:
    return
  while a != b:
    if a > b:
      a -= b
      yield a
    else:
      b -= a
      yield b
```

In [ ]:
```
list(gcd_helper(-18,16))
```

Out[ ]:
```
[]
```

Generator Expressions

Generator expressions look like `list` - `set` comprehensions. But they are completely **different**.

In [ ]:
```
(a*a for a in range(10) if a & 1)
```

Out[ ]:
```
<generator object <genexpr> at 0x7ade317075e0>
```

`list` - `set` comprehensions create a `list` - `set` by immediately evaluating the elements.

In [ ]:
```
{a*a for a in range(10) if a & 1}
```

Out[ ]:
```
{1, 9, 25, 49, 81}
```

In [ ]:
```
a = [a*a for a in range(10) if a & 1]
a.reverse()
a
```

Out[ ]:
```
[81, 49, 25, 9, 1]
```

In [ ]:
```
{a:a*a for a in range(10) if a & 1}
```

Out[ ]:
```
{1: 1, 3: 9, 5: 25, 7: 49, 9: 81}
```

Here, the `if` part is optional and can be ommited like `list` - `set` comprehensions.

In [ ]:
```
squares = (i**2 for i in range(2**256))
for square in squares:
  print(square)
  if square >= 100: break
```

```
0
1
4
9
16
25
36
49
64
81
100
```

Exercises

In [ ]:
```
help(all)
```

```
Help on built-in function all in module builtins:

all(iterable, /)
    Return True if bool(x) is True for all values x in the iterable.

    If the iterable is empty, return True.
```

In [ ]:
```
gen1 = (a**2 for a in range(2**256))
bool(gen1)
```

Out[ ]:
```
True
```

In [ ]:
```
all((gen1,))
```

Out[ ]:
```
True
```

In [ ]:
```
all(a**2 for a in range(2**256))
```

Out[ ]:
```
False
```

In [ ]:
```
help(any)
```

```
Help on built-in function any in module builtins:

any(iterable, /)
    Return True if bool(x) is True for any x in the iterable.

    If the iterable is empty, return False.
```

In [ ]:
```
any(a**2 for a in range(2**256))
```

Out[ ]:
```
True
```

In [ ]:
```
import math

def prime_number_generator():
  n = 2
  while True:
    if all(n % i for i in range(2,math.floor(math.sqrt(n))+1)):
      yield n
    n += 1
```

In [ ]:
```
%%time
primes = prime_number_generator()
```

```
for _ in range(1000): print(next(primes),end=' ')
```

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 107 109
113 127 131 137 139 149 151 157 163 167 173 179 181 191 193 197 199 211 223 227 229 23
3 239 241 251 257 263 269 271 277 281 283 293 307 311 313 317 331 337 347 349 353 359
367 373 379 383 389 397 401 409 419 421 431 433 439 443 449 457 461 463 467 479 487 49
1 499 503 509 521 523 541 547 557 563 569 571 577 587 593 599 601 607 613 617 619 631
641 643 647 653 659 661 673 677 683 691 701 709 719 727 733 739 743 751 757 761 769 77
3 787 797 809 811 821 823 827 829 839 853 857 859 863 877 881 883 887 907 911 919 929
937 941 947 953 967 971 977 983 991 997 1009 1013 1019 1021 1031 1033 1039 1049 1051 1
061 1063 1069 1087 1091 1093 1097 1103 1109 1117 1123 1129 1151 1153 1163 1171 1181 11
87 1193 1201 1213 1217 1223 1229 1231 1237 1249 1259 1277 1279 1283 1289 1291 1297 130
1 1303 1307 1319 1321 1327 1361 1367 1373 1381 1399 1409 1423 1427 1429 1433 1439 1447
1451 1453 1459 1471 1481 1483 1487 1489 1493 1499 1511 1523 1531 1543 1549 1553 1559 1
567 1571 1579 1583 1597 1601 1607 1609 1613 1619 1621 1627 1637 1657 1663 1667 1669 16
93 1697 1699 1709 1721 1723 1733 1741 1747 1753 1759 1777 1783 1787 1789 1801 1811 182
3 1831 1847 1861 1867 1871 1873 1877 1879 1889 1901 1907 1913 1931 1933 1949 1951 1973
1979 1987 1993 1997 1999 2003 2011 2017 2027 2029 2039 2053 2063 2069 2081 2083 2087 2
089 2099 2111 2113 2129 2131 2137 2141 2143 2153 2161 2179 2203 2207 2213 2221 2237 22
39 2243 2251 2267 2269 2273 2281 2287 2293 2297 2309 2311 2333 2339 2341 2347 2351 235
7 2371 2377 2381 2383 2389 2393 2399 2411 2417 2423 2437 2441 2447 2459 2467 2473 2477
2503 2521 2531 2539 2543 2549 2551 2557 2579 2591 2593 2609 2617 2621 2633 2647 2657 2
659 2663 2671 2677 2683 2687 2689 2693 2699 2707 2711 2713 2719 2729 2731 2741 2749 27
53 2767 2777 2789 2791 2797 2801 2803 2819 2833 2837 2843 2851 2857 2861 2879 2887 289
7 2903 2909 2917 2927 2939 2953 2957 2963 2969 2971 2999 3001 3011 3019 3023 3037 3041
3049 3061 3067 3079 3083 3089 3109 3119 3121 3137 3163 3167 3169 3181 3187 3191 3203 3
209 3217 3221 3229 3251 3253 3257 3259 3271 3299 3301 3307 3313 3319 3323 3329 3331 33
43 3347 3359 3361 3371 3373 3389 3391 3407 3413 3433 3449 3457 3461 3463 3467 3469 349
1 3499 3511 3517 3527 3529 3533 3539 3541 3547 3557 3559 3571 3581 3583 3593 3607 3613
3617 3623 3631 3637 3643 3659 3671 3673 3677 3691 3697 3701 3709 3719 3727 3733 3739 3
761 3767 3769 3779 3793 3797 3803 3821 3823 3833 3847 3851 3853 3863 3877 3881 3889 39
07 3911 3917 3919 3923 3929 3931 3943 3947 3967 3989 4001 4003 4007 4013 4019 4021 402
7 4049 4051 4057 4073 4079 4091 4093 4099 4111 4127 4129 4133 4139 4153 4157 4159 4177
4201 4211 4217 4219 4229 4231 4241 4243 4253 4259 4261 4271 4273 4283 4289 4297 4327 4
337 4339 4349 4357 4363 4373 4391 4397 4409 4421 4423 4441 4447 4451 4457 4463 4481 44
83 4493 4507 4513 4517 4519 4523 4547 4549 4561 4567 4583 4591 4597 4603 4621 4637 463
9 4643 4649 4651 4657 4663 4673 4679 4691 4703 4721 4723 4729 4733 4751 4759 4783 4787
4789 4793 4799 4801 4813 4817 4831 4861 4871 4877 4889 4903 4909 4919 4931 4933 4937 4
943 4951 4957 4967 4969 4973 4987 4993 4999 5003 5009 5011 5021 5023 5039 5051 5059 50
77 5081 5087 5099 5101 5107 5113 5119 5147 5153 5167 5171 5179 5189 5197 5209 5227 523
1 5233 5237 5261 5273 5279 5281 5297 5303 5309 5323 5333 5347 5351 5381 5387 5393 5399
5407 5413 5417 5419 5431 5437 5441 5443 5449 5471 5477 5479 5483 5501 5503 5507 5519 5
521 5527 5531 5557 5563 5569 5573 5581 5591 5623 5639 5641 5647 5651 5653 5657 5659 56
69 5683 5689 5693 5701 5711 5717 5737 5741 5743 5749 5779 5783 5791 5801 5807 5813 582
1 5827 5839 5843 5849 5851 5857 5861 5867 5869 5879 5881 5897 5903 5923 5927 5939 5953
5981 5987 6007 6011 6029 6037 6043 6047 6053 6067 6073 6079 6089 6091 6101 6113 6121 6
131 6133 6143 6151 6163 6173 6197 6199 6203 6211 6217 6221 6229 6247 6257 6263 6269 62
71 6277 6287 6299 6301 6311 6317 6323 6329 6337 6343 6353 6359 6361 6367 6373 6379 638
9 6397 6421 6427 6449 6451 6469 6473 6481 6491 6521 6529 6547 6551 6553 6563 6569 6571
6577 6581 6599 6607 6619 6637 6653 6659 6661 6673 6679 6689 6691 6701 6703 6709 6719 6
733 6737 6761 6763 6779 6781 6791 6793 6803 6823 6827 6829 6833 6841 6857 6863 6869 68
71 6883 6899 6907 6911 6917 6947 6949 6959 6961 6967 6971 6977 6983 6991 6997 7001 701
3 7019 7027 7039 7043 7057 7069 7079 7103 7109 7121 7127 7129 7151 7159 7177 7187 7193
7207 7211 7213 7219 7229 7237 7243 7247 7253 7283 7297 7307 7309 7321 7331 7333 7349 7
351 7369 7393 7411 7417 7433 7451 7457 7459 7477 7481 7487 7489 7499 7507 7517 7523 75
29 7537 7541 7547 7549 7559 7561 7573 7577 7583 7589 7591 7603 7607 7621 7639 7643 764
9 7669 7673 7681 7687 7691 7699 7703 7717 7723 7727 7741 7753 7757 7759 7789 7793 7817
7823 7829 7841 7853 7867 7873 7877 7879 7883 7901 7907 7919 CPU times: user 18.1 ms, s
ys: 57 µs, total: 18.2 ms
Wall time: 21.8 ms
```

In [ ]:

```
import math

def prime_number_generator_memoized(t=2**6):
  primes_list = set()
  n=1
```

```
  for n in range(2,t):
    if all(n % i for i in primes_list):
      yield n
      primes_list.add(n)
  while True:
    n += 1
    if all(n % i for i in primes_list) and all(n % i for i in range(max(t,2),math.floo
      yield n
```

In [ ]:
```
%%time
primes = prime_number_generator()
for _ in range(2**16): a=next(primes)
a
```

```
CPU times: user 6.83 s, sys: 29.4 ms, total: 6.86 s
Wall time: 9.87 s
```

Out[ ]: 821641

In [ ]:
```
%%time

primes = prime_number_generator_memoized(t=2**6)
for _ in range(2**16): a=next(primes)
a
```

```
CPU times: user 3.68 s, sys: 10.4 ms, total: 3.69 s
Wall time: 3.7 s
```

Out[ ]: 821641